

Introduction to Structured Query Language

Single Table Queries Part 1

Recall: What is a Database?

- A database stores DATA in TABLES.

The diagram illustrates two database tables: STUDENT and CLASS. Annotations on the left explain the components:

- The STUDENT table**: Points to the STUDENT table header.
- This row stores the data for Sam Cooke**: Points to the first row of the STUDENT table.
- The CLASS table**: Points to the CLASS table header.
- This column stores the ClassName for each class**: Points to the ClassName column in the CLASS table.

STUDENT Table Data:

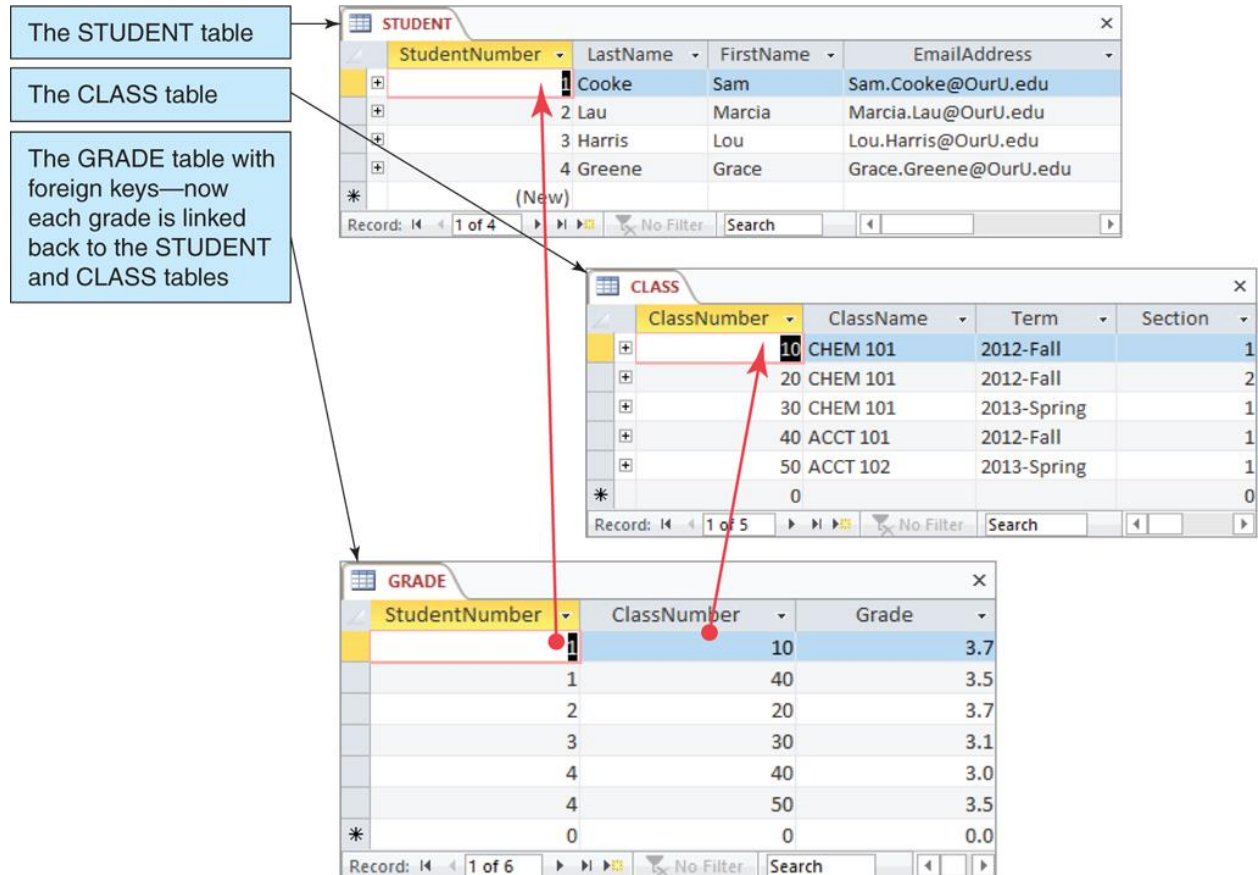
StudentNumber	LastName	FirstName	EmailAddress
1	Cooke	Sam	Sam.Cooke@OurU.edu
2	Lau	Marcia	Marcia.Lau@OurU.edu
3	Harris	Lou	Lou.Harris@OurU.edu
4	Greene	Grace	Grace.Green@OurU.edu
(New)			

CLASS Table Data:

ClassNumber	ClassName	Term	Section
10	CHEM 101	2010-Fall	1
20	CHEM 101	2010-Fall	2
30	CHEM 101	2011-Spring	1
40	ACCT 101	2010-Fall	1
50	ACCT 101	2011-Spring	1

What is a Database?

- The information are saved in different tables which are related to each other.



What is a Database?

- A centralized place to save all the related data.
- Facts
 - Different people use different portions.
 - Different people might use some shared information.

What is a Database?

- Question
 - Will the data saved in Umass Boston this year be the same as the data saved the next year?

What is a Database?

- Question
 - Will the data saved in Umass Boston this year be the same as the data saved the next year?
- Fact
 - Data are not static and will be changed dynamically by different people.



Why do We Need Databases?

- What if we do not have a centralized database, and each person has a copy of the data which is used by him/her?

Why do We Need Databases?

- What if we do not have a centralized database, and each person has a copy of the data which is used by him/her?
- We will have many potential problems to synchronize updates.

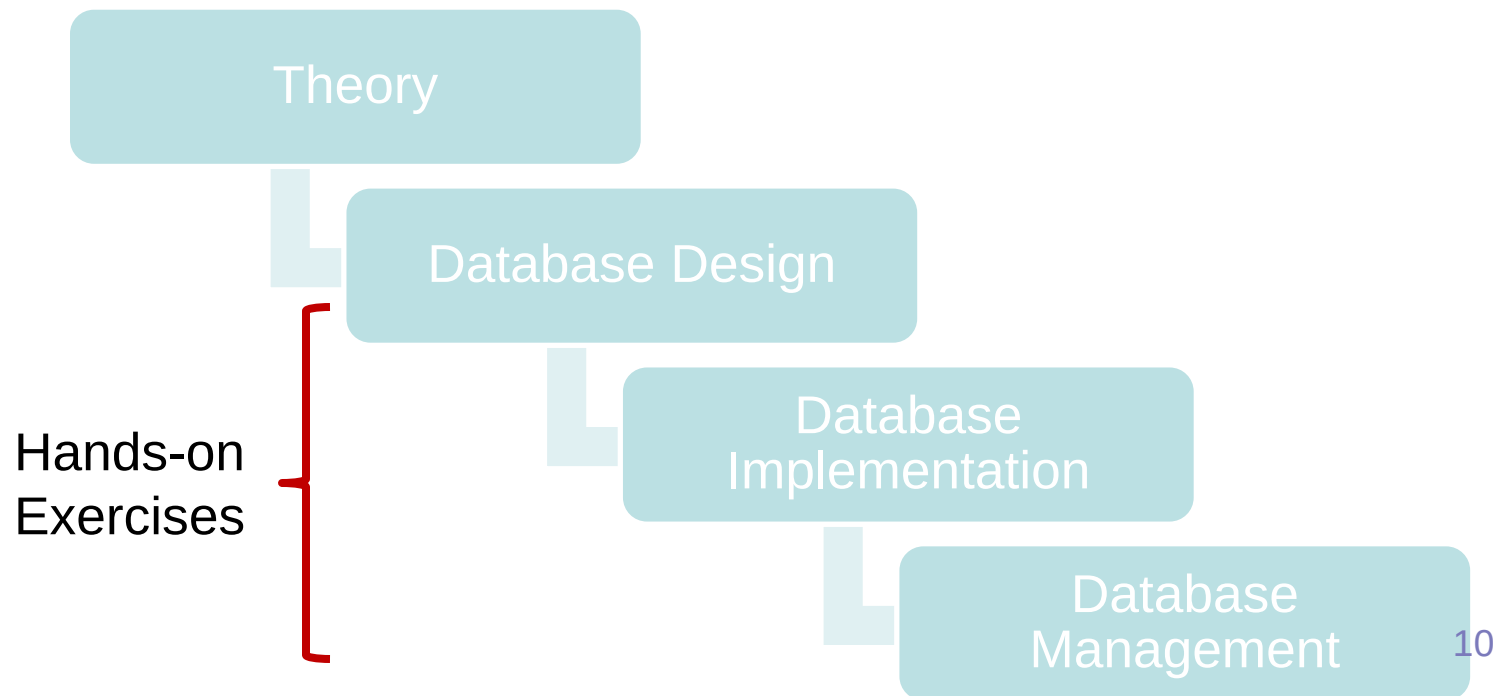
Where to be Used?

- Transaction processing: **transactional database**
 - Purpose: to keep track of things
 - Focus: to make sure each piece of related information is correctly recorded.
- Big data analytics: **data warehouse**
 - Purpose: to gain business insights from historical data
 - Focus: to transfer historical data into certain formats that facilitate big data analytics

Our Goal

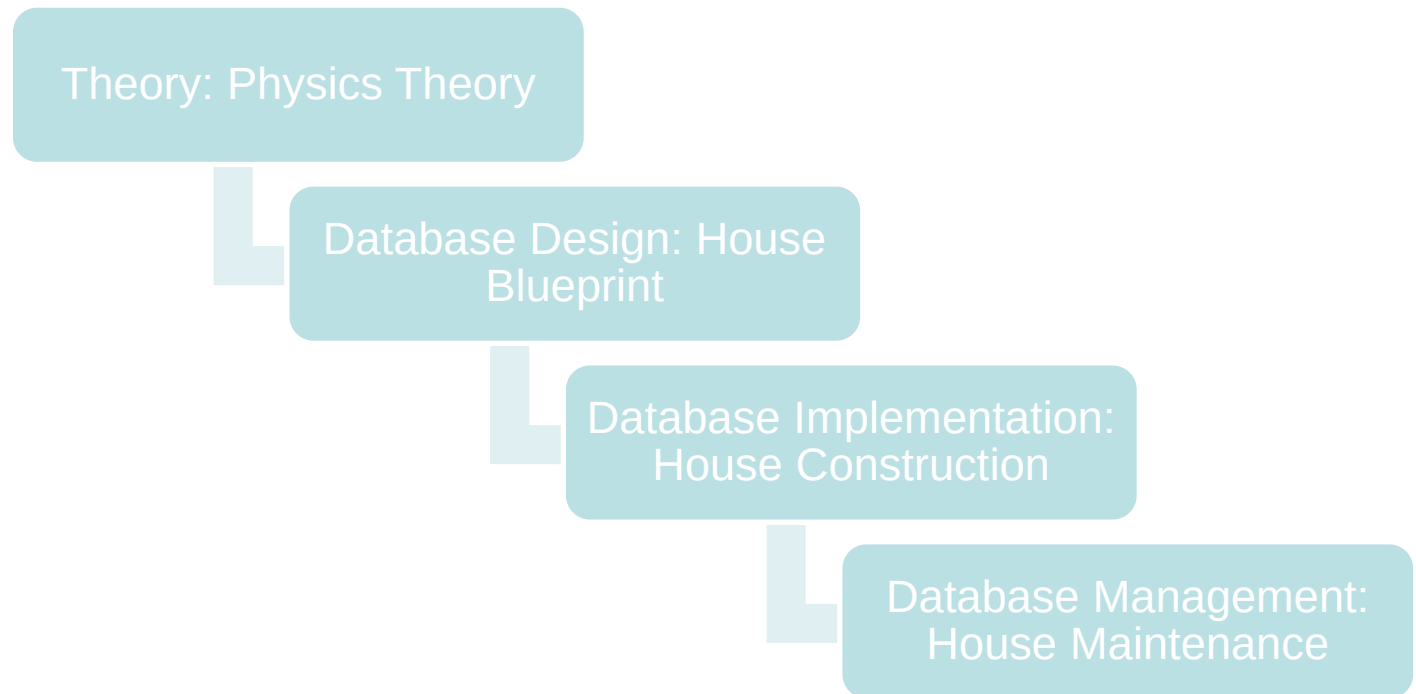
To Understanding Database Management Systems (DBMS)

- **Road Map**

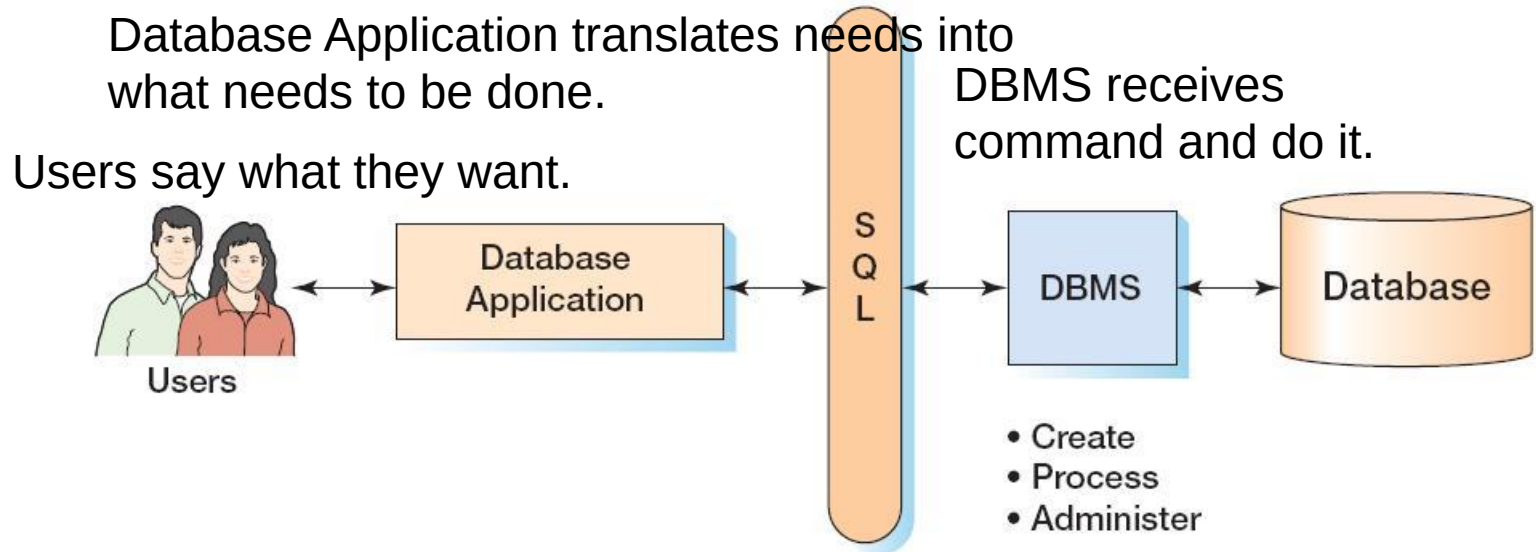


Our Goal

- **Road Map: an analogy to building a house**

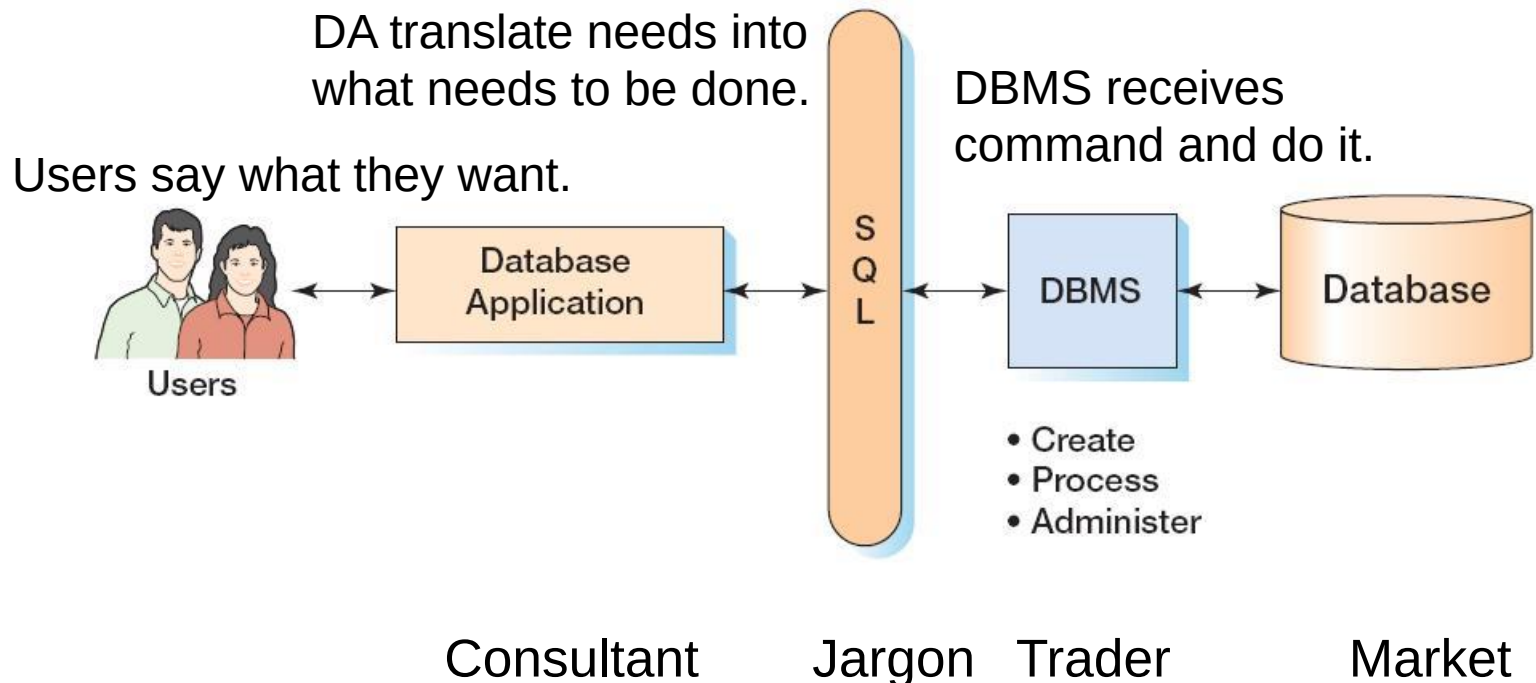


Database Component



Database Component

- An analogy to investment service



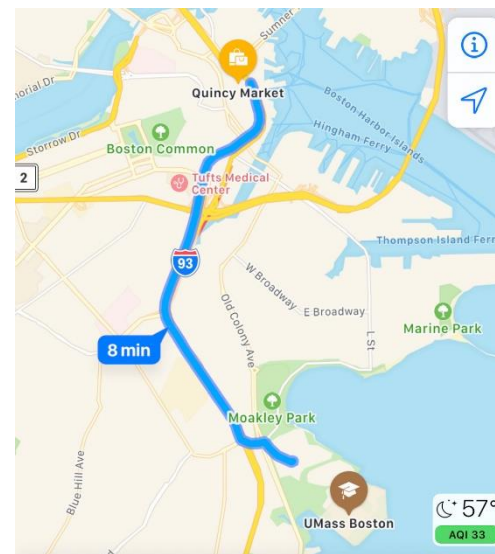
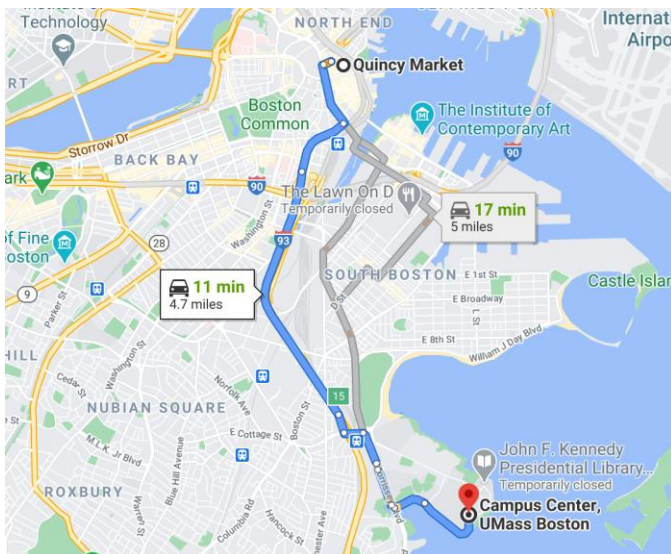
Database Management Systems

- Different database management systems have different ways of store and retrieve data.
 - In Oracle, when you save data, you will save everything in in “[StorageSpaceName].dbf”.
 - In MySQL, you will save table structure in “[DatabaseName].frm” and save data in “[DatabaseName].myd”
 - In SQL Server, you will save the schema and data in “[DatabaseName].mdf” and the log in “[DatabaseName].ldf”

ID	Name	Email
001	Torrie	torrie@umb.edu
002	Dan	dan@umb.edu
003	Mike	mike@umb.edu
004	Bob	bob@umb.edu

Database Management Systems

- Different database management systems have different ways of store and retrieve data.
- Analogy about drive from Quincy Market to UMass Boston



Instructions

- Task: Drive from Quincy Market to UMass Boston



Quincy Market, 206 S Market St, Boston,



Campus Center, UMass Boston, 100 Will



Add destination

Leave now ▾

OPTIONS

 Send directions to your phone

 via I-93 S 11 min
Fastest route, the usual traffic 4.7 miles

[DETAILS](#)

 via Congress St 16 min
4.5 miles

 via Summer St 17 min
5.0 miles

UMass Boston Done

 4 S Market St, Boston, MA 02109, United States

 **0.5 mi**
Keep left to merge onto I-93 South toward Quincy, Worcester

 **0.5 mi**
Keep left to merge onto I-93 South

 **2.0 mi**
Take exit 15 onto Columbia Rd toward S. Boston

 **800 ft**
Turn left onto Columbia Rd toward J.F.K. Library, EMK Institute

 **900 ft**
At the roundabout, take the second exit onto William J Day Blvd

 **250 ft**
Take a sharp right turn onto Mt Vernon St

 **600 ft**
Turn left onto Mt Vernon St

 **0.2 mi**
Turn left

16

Why SQL?

- We need a common language to communicate our goal that is independent from detailed implementation.
- For example:
 - For those students whose GPA is greater than 3, get all their name and school ID information.

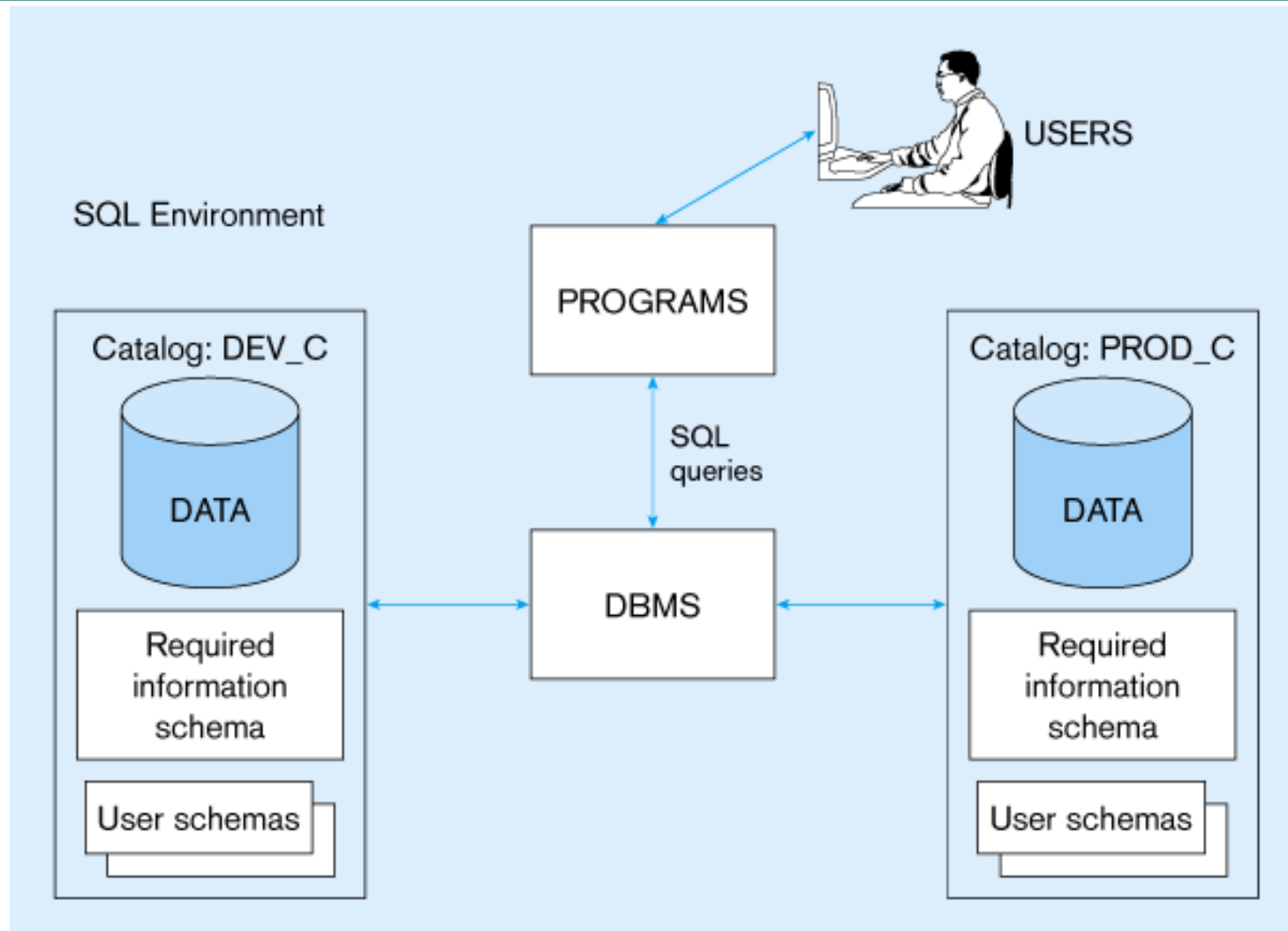
Structured Query Language

- **Structured Query Language (SQL)** was developed by the IBM Corporation in the late 1970's.
- SQL was endorsed as a U.S. national standard by the American National Standards Institute (ANSI) in 1992 [**SQL-92**].

SQL As a Data Sublanguage

- SQL is not a full featured **programming language**.
 - C, C#, Java
- SQL is a **data sublanguage** for creating and processing database data and metadata.
- SQL is ubiquitous in enterprise-class DBMS products.
- SQL programming is a critical skill.

How does SQL work



Schema

- A database **schema** is the blueprint of a database that represents the logical view of the entire database, containing:
 - all the tables
 - all the columns in each table
 - the primary key of each table (indicated by underlining the column names of the primary key columns)
 - and the foreign keys that link the tables together (indicated by italicizing the column names of the foreign key columns)
- Think about Entity Relationship Diagram
- It is the description of database
- It will be specified during database design

SQL Categories

- SQL statements can be divided into five categories:
 - **Data definition language (DDL)**
 - **Data manipulation language (DML)**
 - **SQL/Persistent Stored Modules (SQL/PSM)**
 - **Transaction control language (TCL)**
 - **Data control language (DCL)**

SQL DDL

- **Data definition language (DDL)** statements
 - Used for creating tables, relationships, and other structures

SQL DML

- **Data manipulation language (DML)** statements
 - Used for:
 - Queries – SQL **SELECT** statement
 - Inserting data – SQL **INSERT** statement
 - Modifying data – SQL **UPDATE** statement
 - Deleting data – SQL **DELETE** statement

SQL SQL/PSM

- **SQL/Persistent Stored Modules (SQL/PSM)** statements
 - Add procedural programming capabilities
 - Variables
 - Control-of-flow statements

SQL TCL

- **Transaction control language (TCL)** statements
 - Used to mark transaction boundaries and control transaction behavior

SQL DCL

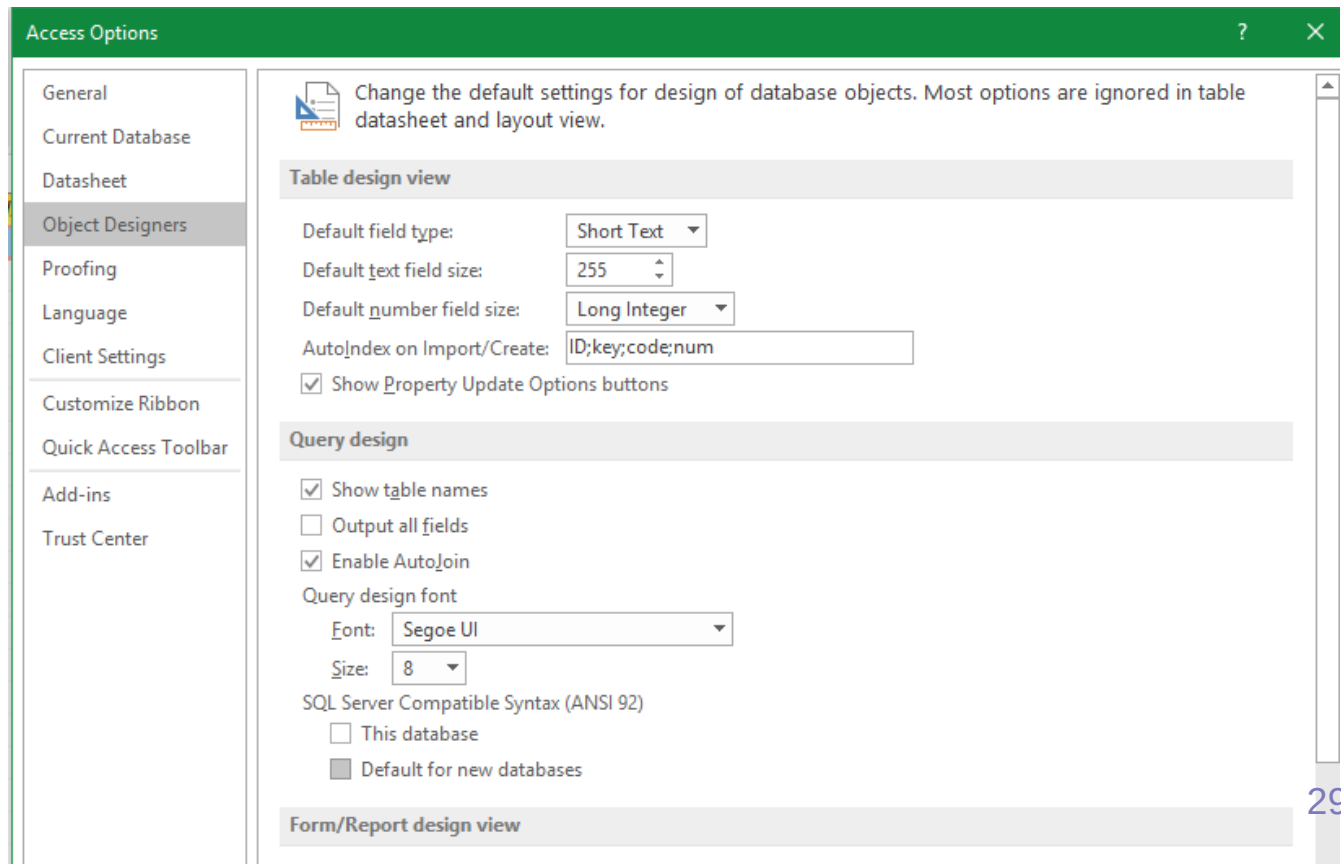
- **Data control language (DCL)** statements
 - Used to grant (or revoke) database permissions to (from) users and groups

Ad-Hoc Queries

- **Ad-hoc queries:**
 - Questions that can be answered using database data
 - Example: “How many customers in Portland, Oregon, bought our green baseball cap?”
 - Created by the user as needed, instead of programmed into an application
 - Common in business

Make Access Compatible to SQL Server (ANSI 92)

- Open Access
- Go to File -> Options -> Object Designers -> SQL Server Compatible Syntax (ANSI 92) -> Check either one based on your need

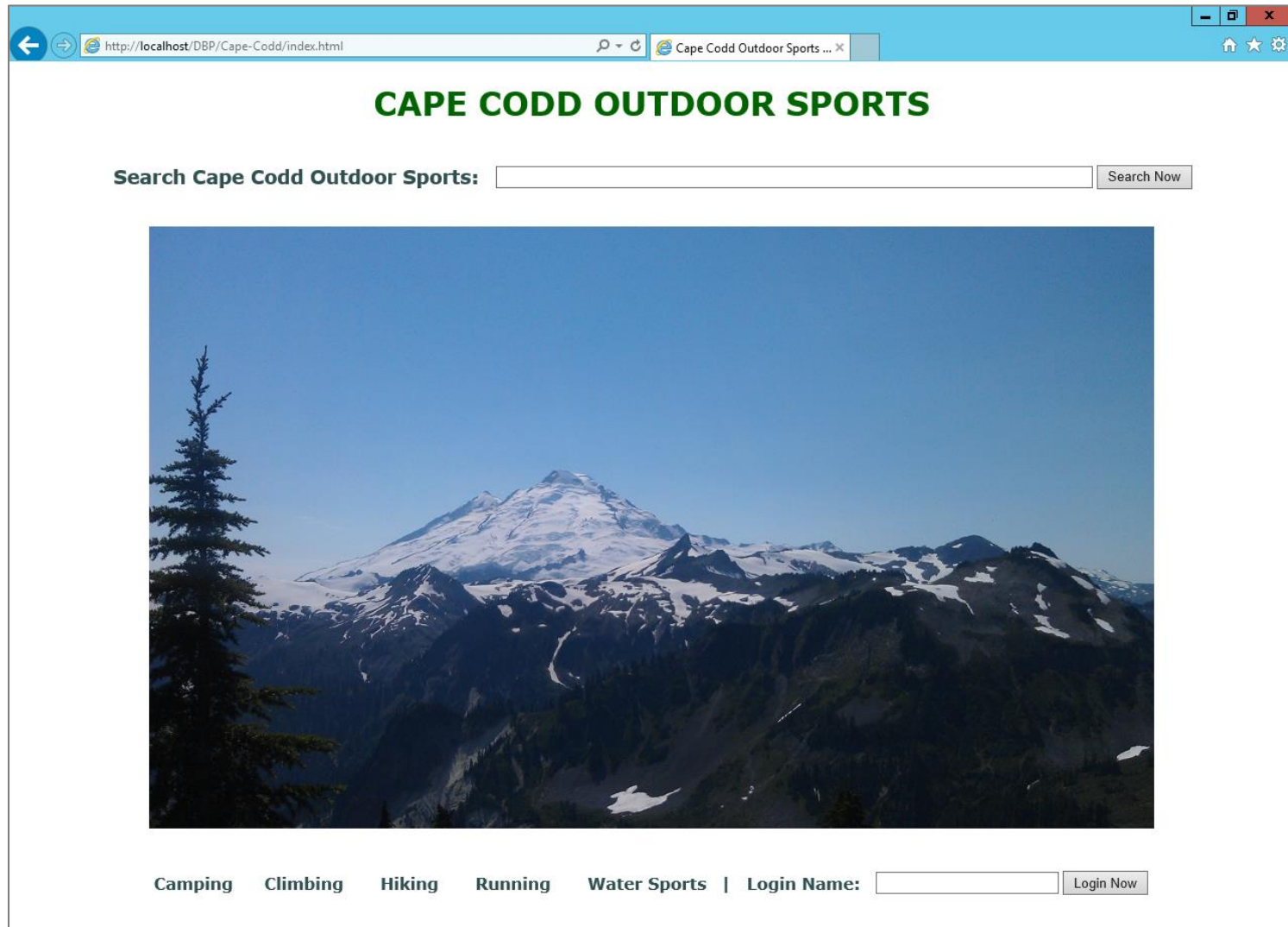


Preparation

- You can install SQL Server Developer and Management Studio directly on Windows (Instructions were given)

Introducing our Sample Database: Cape Codd Outdoor Sports

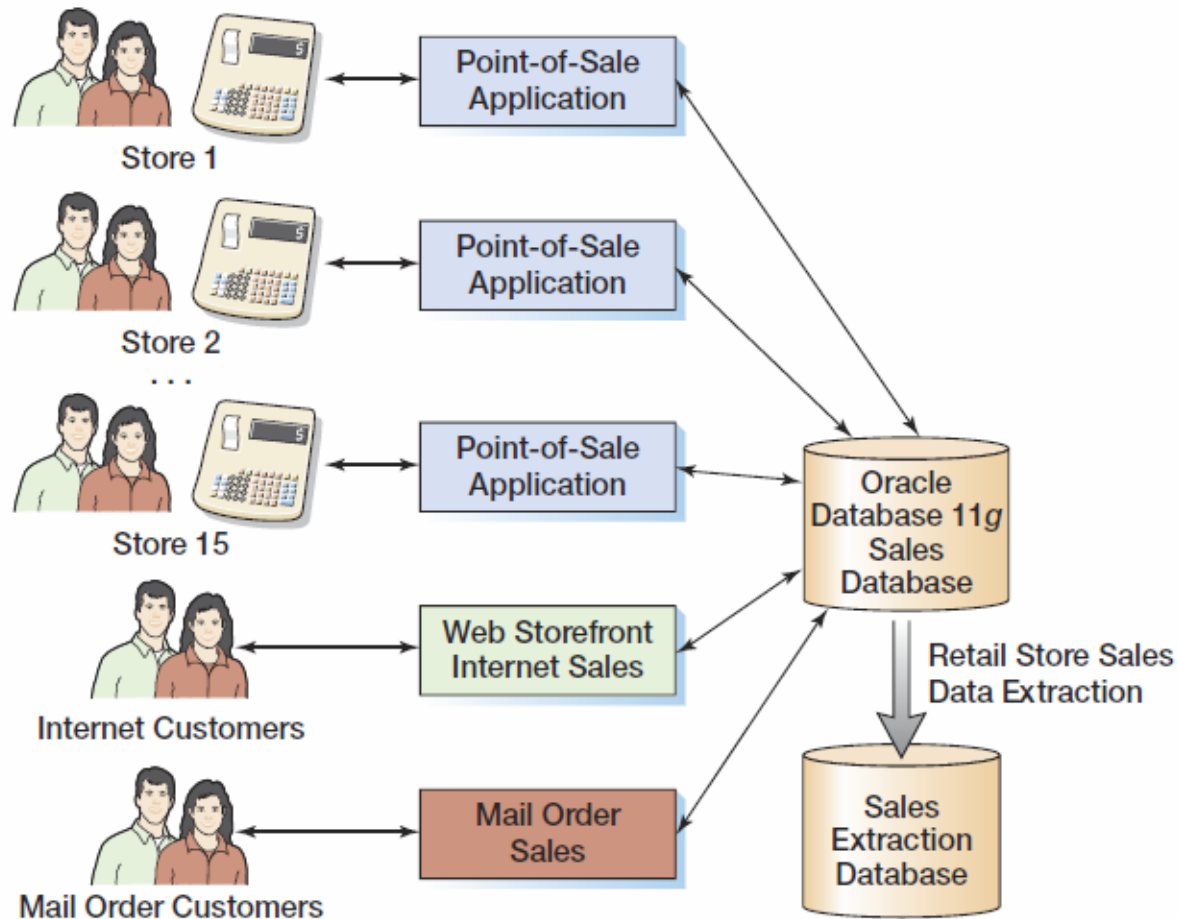
Cape Codd Outdoor Sports



Cape Codd Outdoor Sports

- Cape Codd Outdoor Sports is a fictitious company based on an actual outdoor retail equipment vendor.
- Cape Codd Outdoor Sports:
 - Has 15 retail stores in the United States and Canada.
 - Has an online Internet store.
 - Has a (postal) mail order department.

Cape Codd Retail Sales Structure



Cape Codd Outdoor Sports

Retail Sales and Catalog Context Data Extraction I

- The Cape Codd marketing department needs an analysis of:
 - In-store sales.
 - Catalog content.
- The entire database is not needed for this, only an **extraction** of retail sales data and catalog content.
- The data is extracted by the IS department from the **operational database** into a **separate, off-line database** for use by the marketing department.

Cape Codd Outdoor Sports

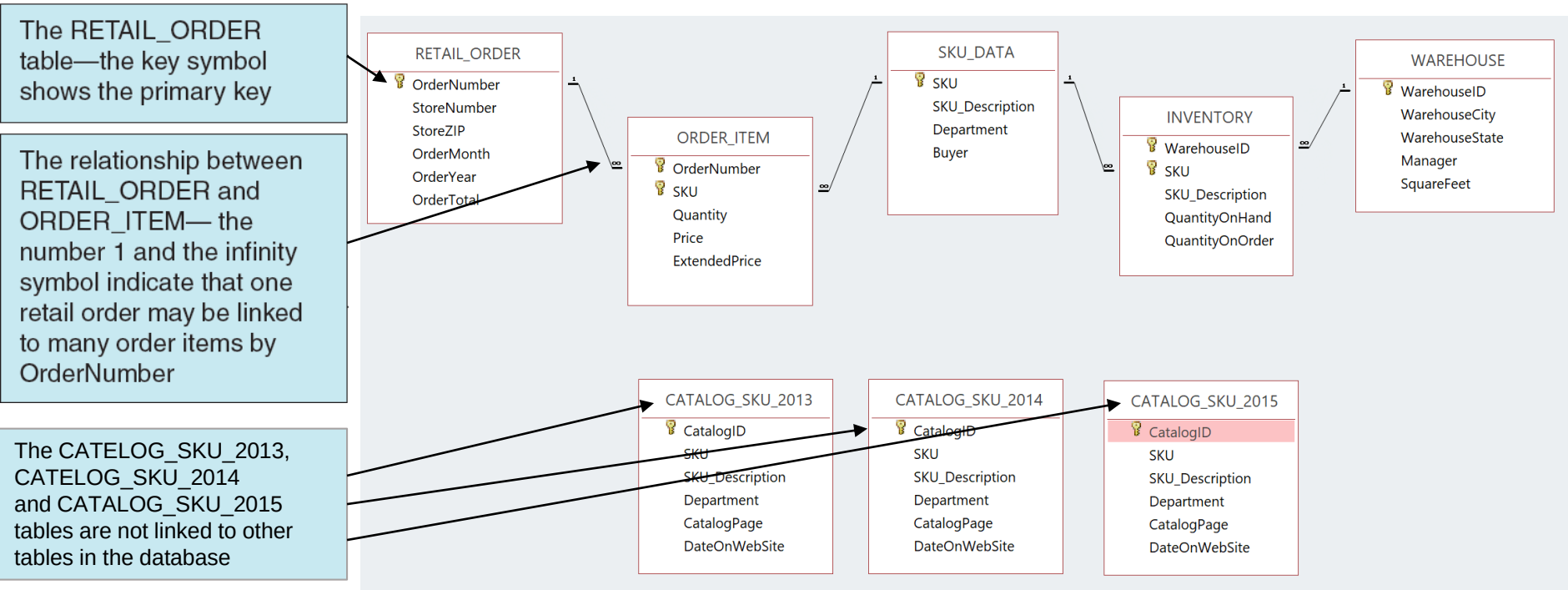
Retail Sales and Catalog Context Data Extraction II

- Five tables are used : **RETAIL_ORDER**, **ORDER_ITEM**, **SKU_DATA** (SKU = Stock Keeping Unit), **CATALOG_SKU_2014**, and **CATALOG_SKU_2015**
- We will also use **CATALOG_SKU_2013**, **WAREHOUSE**, and **INVENTORY** for some examples, and for your assignment

Data Format

Table	Column	Data Type
RETAIL_ORDER	OrderNumber	Integer
	StoreNumber	Integer
	StoreZIP	Character (9)
	OrderMonth	Character (12)
	OrderYear	Integer
	OrderTotal	Currency
ORDER_ITEM	OrderNumber	Integer
	SKU	Integer
	Quantity	Integer
	Price	Currency
	ExtendedPrice	Currency
SKU_DATA	SKU	Integer
	SKU_Description	Character (35)
	Department	Character (30)
	Buyer	Character (30)
CATALOG_SKU_20##	CatalogID	Integer
	SKU	Integer
	SKU_Description	Character (35)
	Department	Character (30)
	CatalogPage	Integer
	DateOnWebSite	Date

Extracted Data Tables I [in Microsoft Access]



Extracted Tables II [in Microsoft Access]

Diagram illustrating the relationships between three tables: RETAIL_ORDER, ORDER_ITEM, and SKU_DATA.

RETAIL_ORDER Table:

OrderNumber	StoreNumber	StoreZIP	OrderMonth	OrderYear	OrderTotal
1000	10	98110	December	2014	\$445.00
2000	20	02335	December	2014	\$310.00
3000	10	98110	January	2015	\$480.00

ORDER_ITEM Table:

OrderNumber	SKU	Quantity	Price	ExtendedPrice
1000	201000	1	\$300.00	\$300.00
1000	202000	1	\$130.00	\$130.00
2000	101100	4	\$50.00	\$200.00
2000	101200	2	\$50.00	\$100.00
3000	100200	1	\$300.00	\$300.00
3000	101100	2	\$50.00	\$100.00
3000	101200	1	\$50.00	\$50.00

SKU_DATA Table:

SKU	SKU_Description	Department	Buyer
100100	Std. Scuba Tank, Yellow	Water Sports	Pete Hansen
100200	Std. Scuba Tank, Magenta	Water Sports	Pete Hansen
101100	Dive Mask, Small Clear	Water Sports	Nancy Meyers
101200	Dive Mask, Med Clear	Water Sports	Nancy Meyers
201000	Half-dome Tent	Camping	Cindy Lo
202000	Half-dome Tent Vestibule	Camping	Cindy Lo
301000	Light Fly Climbing Harness	Climbing	Jerry Martin
302000	Locking Carabiner, Oval	Climbing	Jerry Martin

Extracted Tables III

[in Microsoft Access]

CATALOG_SKU_2014

CATALOG_SKU_2014						
CatalogID	SKU	SKU_Description	Department	CatalogPage	DateOnWebSite	
20140001	100100	Std. Scuba Tank, Yellow	Water Sports	23	1/1/2014	
20140002	100300	Std. Scuba Tank, Light Blue	Water Sports	23	1/1/2014	
20140003	100400	Std. Scuba Tank, Dark Blue	Water Sports		8/1/2014	
20140004	101100	Dive Mask, Small Clear	Water Sports	26	1/1/2014	
20140005	101200	Dive Mask, Med Clear	Water Sports	26	1/1/2014	
20140006	201000	Half-dome Tent	Camping	46	1/1/2014	
20140007	202000	Half-dome Tent Vestibule	Camping	46	1/1/2014	
20140008	301000	Light Fly Climbing Harness	Climbing	77	1/1/2014	
20140009	302000	Locking Carabiner, Oval	Climbing	79	1/1/2014	

CATALOG_SKU_2015

CATALOG_SKU_2015						
CatalogID	SKU	SKU_Description	Department	CatalogPage	DateOnWebSite	
20150001	100100	Std. Scuba Tank, Yellow	Water Sports	23	1/1/2015	
20150002	100200	Std. Scuba Tank, Magenta	Water Sports	23	1/1/2015	
20150003	101100	Dive Mask, Small Clear	Water Sports	27	8/1/2015	
20150004	101200	Dive Mask, Med Clear	Water Sports	27	1/1/2015	
20150005	201000	Half-dome Tent	Camping	45	1/1/2015	
20150006	202000	Half-dome Tent Vestibule	Camping	45	1/1/2015	
20150007	203000	Half-dome Tent Vestibule - Wide	Camping		4/1/2015	
20150008	301000	Light Fly Climbing Harness	Climbing	76	1/1/2015	
20150009	302000	Locking Carabiner, Oval	Climbing	78	1/1/2015	

Using Microsoft SQL Server I

The Microsoft SQL Server Management Studio

The screenshot displays the Microsoft SQL Server Management Studio (SSMS) interface. The main window shows a query titled "SQLQuery1.sql - DJA-CT-NB\SQLSERVER2017.Cape_Codd (DJA-CT-NB\David Auer (58))". The query is as follows:

```
USE Cape_Codd
GO

SELECT SKU, SKU_Description, Department, Buyer
FROM SKU_DATA;
```

The results of the query are displayed in a table with the following columns: SKU, SKU_Description, Department, and Buyer. The results show 13 rows of data.

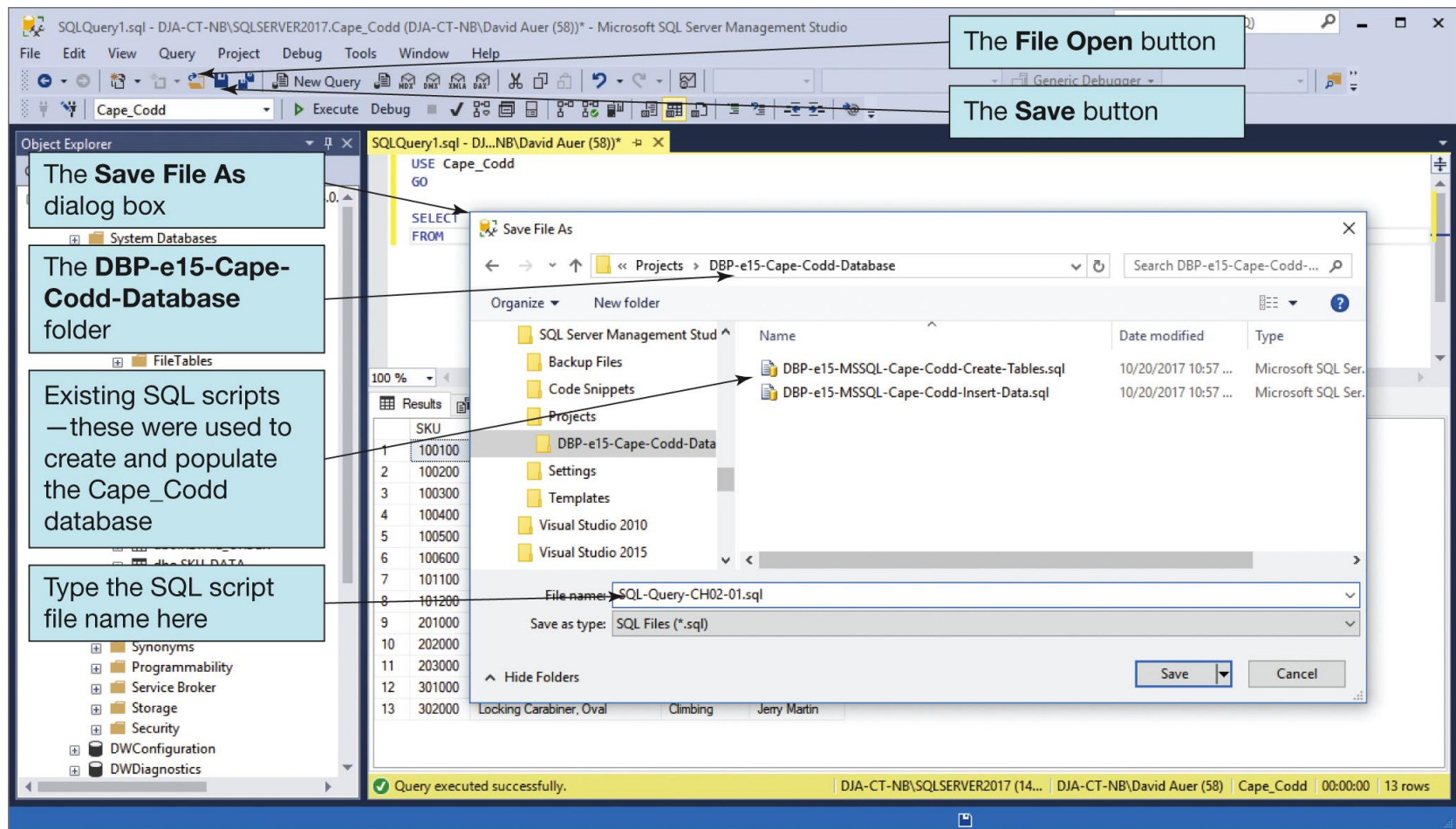
SKU	SKU_Description	Department	Buyer
100100	Std. Scuba Tank, Yellow	Water Sports	Pete Hansen
100200	Std. Scuba Tank, Magenta	Water Sports	Pete Hansen
100300	Std. Scuba Tank, Light Blue	Water Sports	Pete Hansen
100400	Std. Scuba Tank, Dark Blue	Water Sports	Pete Hansen
100500	Std. Scuba Tank, Light Green	Water Sports	Pete Hansen
100600	Std. Scuba Tank, Dark Green	Water Sports	Pete Hansen
101100	Dive Mask, Small Clear	Water Sports	Nancy Meyers
101200	Dive Mask, Med Clear	Water Sports	Nancy Meyers
201000	Half-dome Tent	Camping	Cindy Lo
202000	Half-dome Tent Vestibule	Camping	Cindy Lo
203000	Half-dome Tent Vestibule - Wide	Camping	Cindy Lo
301000	Light Fly Climbing Harness	Climbing	Jerry Martin
302000	Locking Carabiner, Oval	Climbing	Jerry Martin

Callouts identify the following features:

- The **New Query** button
- The **IntelliSense Enabled** button
- The **Parse** button
- The **Execute** button
- The SQL query in the tabbed query window
- The **Available Databases** drop-down list arrow
- The **Cape_Codd** database
- The **Results** tabbed window
- The **Cape_Codd** database tables

Using Microsoft SQL Server II

Saving a Query as an SQL Script in Mgmt Studio



SQL Categories

- Data manipulation language (DML)
 - Queries
 - select
 - Modification
 - insert
 - update
 - delete

SQL SELECT Statements

- Clauses of the SELECT statement:
 - **SELECT**
 - List the columns (and functions) that should be returned from the query
 - **FROM**
 - Indicate the table(s) from which data will be obtained
 - **WHERE**
 - Indicate the conditions under which a row will be included in the result
 - **GROUP BY**
 - Indicate columns to group the results
 - **HAVING**
 - Indicate the conditions under which a group will be included
 - **ORDER BY**
 - Sorts the result according to specified columns

The SQL SELECT Statement

- All SQL statements end with a **semi-colon (;)**.

Specific Columns in One Table

```
SELECT    SKU, SKU_Description, Department, Buyer
FROM      SKU_DATA;
```

	SKU	SKU_Description	Department	Buyer
1	100100	Std. Scuba Tank, Yellow	Water Sports	Pete Hansen
2	100200	Std. Scuba Tank, Magenta	Water Sports	Pete Hansen
3	101100	Dive Mask, Small Clear	Water Sports	Nancy Meyers
4	101200	Dive Mask, Med Clear	Water Sports	Nancy Meyers
5	201000	Half-dome Tent	Camping	Cindy Lo
6	202000	Half-dome Tent Vestibule	Camping	Cindy Lo
7	301000	Light Fly Climbing Harness	Climbing	Jerry Martin
8	302000	Locking Carabiner, Oval	Climbing	Jerry Martin

NOTE

- No difference between upper and lower case.
- No difference between one line and several lines.
- The **square brackets** [] are required if you use reserved keywords or special characters (e.g. space, hyphen) in the column names or identifiers.

Your Turn: Practice

- Find the OrderNumber and OrderTotal of each record in RETAIL_ORDER Table
 - 3 records

Selecting All Columns: The SQL Asterisk (*) Wildcard Character

```
SELECT      *  
FROM        SKU_DATA ;
```

	SKU	SKU_Description	Department	Buyer
1	100100	Std. Scuba Tank, Yellow	Water Sports	Pete Hansen
2	100200	Std. Scuba Tank, Magenta	Water Sports	Pete Hansen
3	101100	Dive Mask, Small Clear	Water Sports	Nancy Meyers
4	101200	Dive Mask, Med Clear	Water Sports	Nancy Meyers
5	201000	Half-dome Tent	Camping	Cindy Lo
6	202000	Half-dome Tent Vestibule	Camping	Cindy Lo
7	301000	Light Fly Climbing Harness	Climbing	Jerry Martin
8	302000	Locking Carabiner, Oval	Climbing	Jerry Martin

Your Turn

- Select all the columns in Table RETAIL_ORDER
 - 3 records

Specifying Column Order I

```
SELECT    Department, Buyer
FROM      SKU_DATA;
```

	Department	Buyer
1	Water Sports	Pete Hansen
2	Water Sports	Pete Hansen
3	Water Sports	Nancy Meyers
4	Water Sports	Nancy Meyers
5	Camping	Cindy Lo
6	Camping	Cindy Lo
7	Climbing	Jerry Martin
8	Climbing	Jerry Martin

Specifying Column Order II

```
SELECT      Buyer, Department
FROM        SKU_DATA;
```

	Buyer	Department
1	Pete Hansen	Water Sports
2	Pete Hansen	Water Sports
3	Nancy Meyers	Water Sports
4	Nancy Meyers	Water Sports
5	Cindy Lo	Camping
6	Cindy Lo	Camping
7	Jerry Martin	Climbing
8	Jerry Martin	Climbing

Your Turn

- Show OrderNumber and OrderTotal in Table RETAIL_ORDER
 - 3 records
- Show OrderTotal and OrderNumber in Table RETAIL_ORDER
 - 3 records

Reading Specified Rows from a Single Table

The SQL DISTINCT Keyword

```
SELECT    DISTINCT Buyer, Department
FROM      SKU_DATA;
```

	Buyer	Department
1	Cindy Lo	Camping
2	Jerry Martin	Climbing
3	Nancy Meyers	Water Sports
4	Pete Hansen	Water Sports

Your Turn

- List the distinct SKU number from SKU_Data table.
 - 8 records

Reading Specified Rows from a Single Table

The SQL TOP {NumberOfRows} Function

```
SELECT      TOP 5 Buyer, Department
FROM        SKU_DATA;
```

	Buyer	Department
1	Pete Hansen	Water Sports
2	Pete Hansen	Water Sports
3	Nancy Meyers	Water Sports
4	Nancy Meyers	Water Sports
5	Cindy Lo	Camping

NOTE: This is an **SQL Server specific** function. See the Oracle Database and MySQL 5.6 documentation for the equivalent functionality in those DBMSs.

Reading Specified Rows from a Single Table

The SQL TOP {Percentage} PERCENT Function

```
SELECT      TOP 75 PERCENT Buyer, Department
FROM        SKU_DATA;
```

	Buyer	Department
1	Pete Hansen	Water Sports
2	Pete Hansen	Water Sports
3	Nancy Meyers	Water Sports
4	Nancy Meyers	Water Sports
5	Cindy Lo	Camping
6	Cindy Lo	Camping

NOTE: This is an SQL Server specific function. See the Oracle Database and MySQL documentation for the equivalent functionality in those DBMSs.

Your Turn

- Show the first two records of OrderNumber and OrderTotal in Table RETAIL_ORDER
 - 2 records
- Show the top 25% records of SKU and Buyer in Table SKU_DATA
 - 2 records

Reading Specified Rows from a Single Table

The SQL WHERE Clause: Character Strings

```
SELECT      *  
FROM        SKU_DATA  
WHERE       Department = 'Water Sports';
```

	SKU	SKU_Description	Department	Buyer
1	100100	Std. Scuba Tank, Yellow	Water Sports	Pete Hansen
2	100200	Std. Scuba Tank, Magenta	Water Sports	Pete Hansen
3	101100	Dive Mask, Small Clear	Water Sports	Nancy Meyers
4	101200	Dive Mask, Med Clear	Water Sports	Nancy Meyers

NOTE: SQL wants a plain ASCII single quote: ' NOT ` !

Your Turn

- Show all the columns from buyer Nancy Meyers in table SKU_DATA
 - 2 records

SQL Comparison Operators

SQL Comparison Operators	
Operator	Meaning
=	Is equal to
<>	Is NOT Equal to
<	Is less than
>	Is greater than
<=	Is less than OR equal to
>=	Is greater than OR equal to
IN	Is equal to one of a set of values
NOT IN	Is NOT Equal to one of a set of values
BETWEEN	Is within a range of numbers (includes the end points)
NOT BETWEEN	Is NOT within a range of numbers (includes the end points)
LIKE	Matches a set of characters
NOT LIKE	Does NOT match a set of characters
IS NULL	Is equal to NULL
IS NOT NULL	Is NOT equal to NULL

Reading Specified Rows from a Single Table

The SQL WHERE Clause: Dates

```
SELECT      *  
  
FROM        CATALOG_SKU_2014  
  
WHERE       DateOnWebSite = '01-JAN-2014';
```

	CatalogID	SKU	SKU_Description	Department	CatalogPage	DateOnWebSite
1	20140001	100100	Std. Scuba Tank, Yellow	Water Sports	23	2014-01-01
2	20140002	100300	Std. Scuba Tank, Light Blue	Water Sports	23	2014-01-01
3	20140004	101100	Dive Mask, Small Clear	Water Sports	26	2014-01-01
4	20140005	101200	Dive Mask, Med Clear	Water Sports	26	2014-01-01
5	20140006	201000	Half-dome Tent	Camping	46	2014-01-01
6	20140007	202000	Half-dome Tent Vestibule	Camping	46	2014-01-01
7	20140008	301000	Light Fly Climbing Harness	Climbing	77	2014-01-01
8	20140009	302000	Locking Carabiner, Oval	Climbing	79	2014-01-01

NOTE: Microsoft Access dates must be enclosed within # symbols: #01/01/14#

Your Turn

- Show SKUs that are on Website after March 1st 2015 from Catalog_SKU_2015 table
 - 1 record

Reading Specified Rows from a Single Table

The SQL WHERE Clause: Numbers

```
SELECT      *  
FROM        SKU_DATA  
WHERE       SKU > 200000;
```

	SKU	SKU_Description	Department	Buyer
1	201000	Half-dome Tent	Camping	Cindy Lo
2	202000	Half-dome Tent Vestibule	Camping	Cindy Lo
3	301000	Light Fly Climbing Hamess	Climbing	Jery Martin
4	302000	Locking Carabiner, Oval	Climbing	Jery Martin

Your Turn

- Show all columns from inventory table where SKU is no less than 300000

Reading Specified Columns and Rows from a Single Table

Column Names and the SQL WHERE Clause

```
SELECT      SKU_Description, Department
FROM        SKU_DATA
WHERE       Department = 'Climbing';
```

	SKU_Description	Department
1	Light Fly Climbing Harness	Climbing
2	Locking Carabiner, Oval	Climbing

NOTE: A column used in the WHERE clause does **not** have to be one of the columns listed in the SELECT clause.

Your Turn

- Show Department, SKU_Description and Buyer information of Camping Department from SKU_DATA table
 - 2 records

Sorting the SQL Query Results

The SQL ORDER BY Clause

```
SELECT      *  
FROM        ORDER_ITEM  
ORDER BY    OrderNumber;
```

	OrderNumber	SKU	Quantity	Price	ExtendedPrice
1	1000	201000	1	300.00	300.00
2	1000	202000	1	130.00	130.00
3	2000	101100	4	50.00	200.00
4	2000	101200	2	50.00	100.00
5	3000	101200	1	50.00	50.00
6	3000	101100	2	50.00	100.00
7	3000	100200	1	300.00	300.00

Your Turn

- Select all the columns from Order_Item table, order the records based on order number and price.
- Select all the columns from Order_Item table, order the records based on price and order number.

Sorting the SQL Query Results

Ascending and Descending Sort Order

```
SELECT      *  
FROM        ORDER_ITEM  
ORDER BY    Price DESC, OrderNumber ASC;
```

OrderNumber ▾	SKU ▾	Quantity ▾	Price ▾	ExtendedPrice ▾
1000	201000	1	\$300.00	\$300.00
3000	100200	1	\$300.00	\$300.00
1000	202000	1	\$130.00	\$130.00
2000	101200	2	\$50.00	\$100.00
2000	101100	4	\$50.00	\$200.00
3000	101200	1	\$50.00	\$50.00
3000	101100	2	\$50.00	\$100.00

NOTE: The default sort order is ASC—it does not have to be specified.

Your Turn

- Select all the columns from Order_Item table, order the records the following way: an ascending order based on price and a descending order based on OrderNumber.